

Module 9 Week A — Applied Lab: KG Construction

Due: Tuesday end of day. Resubmissions are accepted through Saturday of the assignment week.

The lab takes the RDF, Turtle, Fuseki, and SPARQL material from the reading and turns it into an end-to-end stack you stand up yourself. You will model a small domain as an ontology, serialize it as Turtle, run Apache Jena Fuseki via Docker, load the dataset, and verify it with three SPARQL queries — once against the local TTL file with `rdflib` and once against the live Fuseki endpoint with `SPARQLWrapper`.

By the end of the lab you have closed the gap between “I can read a SPARQL query” and “I can stand up a triple store, populate it, and query it.”

Setup

Submission flow — all Module 9 Week A assignments. Module 9 Week A uses a fork-and-submit flow (drill, lab, integration) instead of GitHub Classroom. GitHub is sunsetting Classroom (full transition 2026-08-28). Steps 1–3 below are the fork-and-submit part; the rest of the steps are the same workflow you used in prior modules.

If you haven't already installed the M9 packages (Python + Fuseki Docker image), go back to the [Setup: install before the concept session](#) section of the reading and complete the install before proceeding.

- Fork the template repo to your GitHub account.** Open github.com/LevelUp-Applied-AI/m9-l9a and click **Fork** (top right) → set the owner to your personal GitHub account → click **Create fork**. Leave “Copy the main branch only” checked.
- Enable Actions on your fork.** On your forked repo, click the **Actions** tab. GitHub disables workflows on forks by default — click **I understand my workflows, go ahead and enable them**. (One-time per fork. The autograder cannot run on your PR until this is done.)
- Clone *your* fork (not the upstream repo):**

```
git clone https://github.com/<your-github-username>/m9-l9a.git
cd m9-l9a
```

- Install dependencies (cached from the drill — should be fast):** `pip install -r requirements.txt`
- Bring up Fuseki:** `docker compose up -d`. Wait until `curl -s http://localhost:3030/$/ping` returns a timestamp.
- Create your working branch:** `git checkout -b lab-kg-construction`

The domain — recipes

You are modeling a small **recipes** ontology — ~15 recipes, each with a name, a cuisine (e.g., Italian, Thai), a primary ingredient, an author, and a publication year. Cuisines and primary ingredients are themselves named classes that carry SKOS labels (e.g., `:Italian` has `skos:prefLabel` “Italian” and `skos:altLabel` “italiano”).

The starter ships with `recipes_partial.ttl` containing **5 recipes pre-filled** plus the class declarations for two cuisines and three primary ingredients. Your job is to extend it to **15 recipes** following the same pattern, plus all SKOS labels.

Your tasks

- Extend `recipes_partial.ttl` to 15 recipes

Add 10 more recipes following the existing pattern. Each recipe needs:

- `:name` — a string literal
- `:cuisine` — a reference to a cuisine class (e.g., `:Italian`)
- `:primaryIngredient` — a reference to an ingredient class (e.g., `:eggplant`)
- `:author` — a reference to an author resource
- `:year` — an integer literal

Include a mix of years from 2015 through 2024 so the post-2020 query has interesting boundaries.

Each cuisine and ingredient class must carry both `skos:prefLabel` (its canonical name) and at least one `skos:altLabel` (an alternative — e.g., “italiano” for `:Italian`, “aubergine” for `:eggplant`). One of your 15 recipes must use a cuisine *whose label match only succeeds via* `skos:altLabel` — the autograder verifies that `q2` returns it.

- Verify the file parses

Before bringing up Fuseki, sanity-check the TTL:

```
python -c "from rdflib import Graph; g = Graph(); g.parse('recipes_partial.ttl',
```

You should see a triple count in the ~100–130 range — roughly 90 from the 15 recipes (5 required properties each, plus the `a :Recipe` type triple) plus your cuisine / ingredient / author class triples (each typically contributes 2–3 triples). The autograder requires at least 60 triples as a floor; the exact number depends on how many cuisines and ingredients you add. A parse error points at the line.

- Load the dataset into Fuseki

Complete `load_dataset.py`. The starter has the TODOs marked:

```
# load_dataset.py — your task is to fill in the POST request

import os
import requests

FUSEKI_DATA_URL = "http://localhost:3030/recipes/data"
TTL_FILE = "recipes_partial.ttl"
FUSEKI_USER = os.getenv("FUSEKI_USER", "admin")
FUSEKI_PASSWORD = os.getenv("FUSEKI_PASSWORD", "admin")

def main():
    # TODO: open TTL_FILE in binary mode and POST to FUSEKI_DATA_URL
    # with Content-Type: text/turtle and HTTP Basic Auth.
    # Raise on non-2xx.
    ...

if __name__ == "__main__":
    main()
```

Why the auth? `docker-compose.yml` starts Fuseki with `ADMIN_PASSWORD: admin`, which gates write endpoints (including `POST /recipes/data`) behind HTTP Basic Auth. The defaults in the starter match the compose file, so no env vars are needed locally. If you change the compose password, override via `FUSEKI_USER` / `FUSEKI_PASSWORD` env vars. The autograder’s CI uses the same defaults — no extra configuration on your side.

Run it:

```
python load_dataset.py
```

Then confirm the data is in Fuseki:

```
curl -G http://localhost:3030/recipes/sparql \
--data-urlencode "query=ASK { ?s ?p ?o }" \
-H "Accept: application/json"
```

You should see `{ "head": {}, "boolean": true }`.

- Write three SPARQL queries

Complete `queries.py`. Three functions, each returning a SPARQL string.

```
# queries.py — your task is to fill in the SPARQL string each function returns

def q1():
    """
    Q1 — List all recipes and their names.

    Variables in the SELECT: ?recipe ?name.
    Result: 15 rows.
    """
    return "" # TODO

def q2():
    """
    Q2 — List all Italian recipes — matching either skos:prefLabel "Italian"
    OR skos:altLabel "italiano". Use the SKOS disambiguation pattern from
    the reading §7. The query must NOT silently miss recipes whose cuisine
    is only matched via skos:altLabel.

    Variables in the SELECT: ?recipe ?name.
    """
    return "" # TODO

def q3():
    """
    Q3 — List recipes published after 2020 with their primary ingredient.
    Return the ingredient resource (URI), not its label.

    Variables in the SELECT: ?recipe ?ingredient.
    Use FILTER (?year > 2020).
    """
    return "" # TODO
```

The autograder runs each query twice — first against an in-memory `rdflib.Graph` loaded from your TTL file, then against the live Fuseki endpoint via `SPARQLWrapper`. Both must agree.

- Run the autograder locally

```
pytest tests/ -v
```

If a query returns zero rows when you expect rows, re-read the Troubleshooting section of the reading. The two most common culprits are a missing `PREFIX` declaration and a typo in a predicate IRI.

Submission

- Commit and push your branch to *your fork*:

```
git add .
git commit -m "Lab 9A: KG construction"
git push origin lab-kg-construction
```

- Open a Pull Request *within your fork*** — base branch `main`, compare branch `lab-kg-construction`. **Important:** the PR must stay inside your fork. When you click “Compare & pull request”, GitHub defaults to “base repository: LevelUp-Applied-AI/m9-l9a” — change the **base repository** dropdown to your own fork (`<your-github-username>/m9-l9a`) before opening the PR. The autograder runs against your branch; check the CI status on the PR page to confirm green. If you see “Workflows are disabled” on the Actions tab, you missed Setup step 2 — enable them and push an empty commit (`git commit --allow-empty -m "ci: trigger"`) to re-run.
- Your PR description must include:
 - A one-paragraph summary of your ontology design — what classes you used and one decision that was not obvious.
 - A note on any `skos:altLabel`s you added beyond the required minimum.
 - Paste your PR URL into TalentLMS → Module 9 → Lab 9A to submit this assignment.**

Challenge Extensions

These are optional extensions that build on the base lab. They are not graded, but they are exactly the kind of work that prepares you for the integration task and Week B.

Challenge Tier 1 — Subclass entailment

Add a `rdfs:subClassOf` hierarchy to your cuisines. For example: `:Italian rdfs:subClassOf :European .` and `:French rdfs:subClassOf :European .` Write a fourth query that returns all European recipes by walking the subclass relationship explicitly:

```
SELECT ?recipe WHERE {
  ?recipe :cuisine ?c .
  ?c rdfs:subClassOf* :European .
}
```

The `*` is a SPARQL property path — it matches zero or more `rdfs:subClassOf` steps. New concept: SPARQL property paths as a graph-traversal primitive.

Challenge Tier 2 — Bake the dataset into the image

Replace the load-after-start pattern with a `Dockerfile` that pre-loads `recipes_partial.ttl` at container build time, so the dataset is present immediately on `docker run`. Document the trade-offs vs. the load-after-start approach in a `CHALLENGE_TIER2.md` file.

Challenge Tier 3 — VoID descriptor + CONSTRUCT

Author a small **VoID** (Vocabulary of Interlinked Datasets) descriptor for the recipes dataset — `recipes.void.ttl` — describing the dataset’s namespace, triple count, and class partitions. Host it alongside Fuseki. Then write a CONSTRUCT query that emits a summary subgraph linking each recipe to its cuisine and primary ingredient, suitable for a downstream consumer to ingest. Document the design in `CHALLENGE_TIER3.md`.

